

Lab1 五级流水 RISC-V CPU 实现报告

1 实现目标与约束

Lab1 的目标是在给定的框架下实现一个 64 位 RISC-V CPU 核心，采用经典的五级流水结构，即 IF 取指、ID 译码、EX 执行、MEM 访存和 WB 回写五个阶段。处理器需要通过 `make test-lab1`，在 DiffTest 对比下达到 HIT GOOD TRAP，同时实现 Lab1 要求的基础整数指令，包括 `addi/xori/ori/andi` 等立即数与逻辑指令，`add/sub/and/or/xor` 等寄存器算术逻辑指令，以及 `addiw/addw/subw` 等 32 位算术指令。流水线的数据相关处理以转发为主，优先从 EX 和 MEM 阶段前递结果，只有在确实无法前递时才依赖寄存器堆中已经提交的值。分支和跳转采用静态预测策略，目前实现为简单的 Always-Not-Taken，并在真正执行阶段检测错预测后对流水线进行冲刷。核心需要与 DiffTest 框架正确对接，保证提交时刻的 PC、指令和寄存器状态与参考模型严格一致，同时为多周期乘除法指令预留接口，但未实现其具体功能。

2 总体设计思路

整体设计在 `vsrc/src/` 目录下采用模块化拆分的方式组织代码。`core.sv` 作为顶层模块，负责各个流水级之间的连接、全局控制信号以及与 DiffTest 的接口接线；`fetch.sv` 只关注取指逻辑和与 `ibus` 的握手；`decode.sv` 承担指令译码和控制信号生成的职责；`execute.sv` 完成 ALU 运算、分支条件判断和目标 PC 计算；`mem.sv` 在 Lab1 阶段主要做直通，为后续 `load/store` 扩展预留插入点；`writeback.sv` 则把来自 MEM 阶段的结果规范化为统一的 WB 接口信号。寄存器堆被单独实现为 `regfile.sv`，提供两个组合读端口和一个同步写端口；`predictor_static.sv` 是一个简单的静态分支预测器模块，后续可以无缝替换为更复杂的预测结构；`muldiv_stub.sv` 则是多周期乘除法单元的占位模块，用于在架构层面先把握好接口与握手时序。

在流水线内部，`core.sv` 中定义了 `if_id_t`、`id_ex_t`、`ex_mem_t` 和 `mem_wb_t` 四个结构化流水寄存器类型，每一级都带有 `valid` 位，用于标记该级携带的是否为一条真实指令。`valid` 既用于在分支错预测时冲刷年轻指令，也用于在 `trap` 或 `halt` 后快速将整条流水线清空，从而避免无效的 NOP 指令被误当作真正的已提交指令。这种结构化的写法使得信号含义更加清晰，也方便后续在各级扩展新的控制位和附加信息。

3 各流水级的具体实现

在取指阶段，`fetch.sv` 通过维护 `pending` 和 `req_pc` 两个状态，实现了对 `ibus` 时序约束的遵守。每当需要发起新的取指请求时，模块将 `ireq.valid` 置为 1，并将 `ireq.addr` 设为当前请求 PC。只要 `iresp.data_ok` 尚未到来，`ireq` 中的 `valid` 和 `addr` 就会保持稳定不变，保证缓存侧能够可靠接收和处理该请求。当 `data_ok` 变为 1 时，取指模块在本地锁存返回的指令，将其连同 PC 一并输出给上游，并根据是否收到分支重定向信号选择下一个请求 PC：如果有分支修正，则使用 `redirect_pc`，否则就按顺序加 4 继续发起请求。如果全局的 `cpu_halt` 信号被拉高，则取指阶段会停止发起新的请求，从根源上阻断后续指令进入流水。

在译码阶段，`decode.sv` 从 IF/ID 寄存器中读出 PC 和指令，对 RISC-V 指令进行字段解析，生成源寄存器 `rs1`、`rs2`、目的寄存器 `rd` 以及对应的立即数，同时产生 ALU 操作类型、是否为分支或跳转、是否需要写回、是否为 32 位运算 (`is_word`) 等控制信号。为了与 Difftest 和退出机制配合，译码中还识别了两类 trap 指令：一类是标准的 `ebreak`（指令编码 0x00100073），另一类是实验框架中用于结束程序的 `nemu_trap` 风格指令，这里通过识别特定 opcode（例如 7'b1101011 对应的 0x0005006b）将其统一打上 `is_trap` 标记，便于后续在 WB 阶段执行收尾逻辑。

执行阶段由 `execute.sv` 实现。该模块首先根据 `alu_op` 选择执行 `add/sub/and/or/xor` 等基本算术逻辑指令，并在需要时对 32 位结果进行符号扩展，得到正确的 64 位写回值。同时，通过比较操作数、结合 `funct3` 类型来判断分支是否应当被采纳，从而确定 `branch_taken` 和 `branch_target`，其中分支目标为 `pc + imm`，JAL 也是 `pc + imm`，而 JALR 的目标为 `(rs1 + imm) & ~1`。模块将实际的控制流结果与预测结果进行比较，如果预测错误，就发出 `branch_mispredict` 信号，并给出正确的 `branch_correct_pc`，供 IF 阶段重定向。当前的分支预测器 `predictor_static.sv` 实现的是简单的 Always-Not-Taken 策略，接口上已经为未来的 BHT/BTB 等动态预测结构预留了替换空间。

MEM 与 WB 两个阶段主要起到解耦和接线的作用。`mem.sv` 在 Lab1 中尚未承载实际的访存功能，因此只是把 EX 输出的各种信号有损耗地传递到下一阶段，同时保留了形如“在此接入 load/store 请求与旁路”的注释，方便后续在这一位置插入数据缓存或总线接口。

`writeback.sv` 则统一整理来自 MEM 的结果，形成 `wb_valid`、`wb_wen`、`wb_wdest`、`wb_wdata` 以及 `wb_is_trap` 等规范信号，为 commit 寄存器和 Difftest 提供清晰的提交边界。

4 RAW 相关与转发策略

为了减少流水停顿，本设计在 ID 与 EX 之间实现了转发优先的 RAW 相关处理。在 `core.sv` 中，ID 阶段读出寄存器堆的值后，会在组合逻辑中根据目的寄存器编号和写回使能情况，优先尝试从 EX 阶段的结果前递，如果 EX 阶段没有匹配的写回，再尝试从 MEM 阶段前递，最后才使用寄存器堆中已经提交的值。这样的优先顺序保证了最近产生的结果总能尽快被后继指令消费，从而避免了大量的 ALU-ALU 相关导致的气泡。对于 `x0` 寄存器，设计始终将其视为常数 0，不会把写入 `x0` 的操作当作有效前递源，也不会从上游阶段前递写入 `x0` 的结果，符合 RISC-V 规范的语义约束。

5 Difftest 对齐与 trap 收尾策略

在与 Difftest 进行对比时，关键在于“提交点”的时序是否与参考模型严格对齐。有出现“GOOD TRAP 之后还多提交了 1-2 条 NOP 指令”以及 `this_pc` 报错的信息，与提交时序和 trap 收尾处理有关。

为了解决时序上的 off-by-one 问题，`core.sv` 中引入了一层独立的 commit 寄存器。WB 阶段的 `wb_valid`、`wb_pc`、`wb_instr`、`wb_wen`、`wb_wdest` 和 `wb_wdata` 等信号不会直接接给 `DifftestInstrCommit`，而是在时钟上升沿被打一拍，存入 `commit_valid`、`commit_pc`、`commit_instr`、`commit_wen`、`commit_wdest` 和 `commit_wdata`。Difftest 始终以 commit 寄存器中的内容作为“本拍刚刚提交完成的指令”进行比对，从而消除了 WB 组合逻辑与寄存器堆写入之间的时序错位。

同时，为了让 Difftest 看到的寄存器状态与 commit 信息对应，`regfile.sv` 一方面通过 `next_reg` 导出“下一拍将要写入”的寄存器值，另一方面通过 `reg_state` 导出当前拍真实存在于寄存器中的架构状态。`DifftestArchIntRegState` 读取的是 `reg_state` 而不是 `next_reg`，这样在某一拍 `commit_valid` 为 1 时，Difftest 读取到的寄存器内容正好反映了前一拍 WB 写入已经完成之后的结果，与 commit 的 PC 和指令语义保持同步。

trap 收尾策略是另一个重要点。当 trap 指令（包括 `ebreak` 和 `nemu_trap`）到达 WB 阶段时，它首先会像普通指令一样进入 commit 寄存器，保证 Difftest 能看见这条最后的提交。在下一拍，核心根据 WB 阶段的 `wb_is_trap` 标志拉起全局的 `cpu_halt` 信号，并将 IF/ID/EX/MEM/WB 几个流水寄存器中的 `valid` 统一清零，彻底清空整条流水线。与此同时，`commit_valid` 也通过 `!cpu_halt` 进行遮蔽，一旦停机后就不再产生新的提交记录。这样 trap 指令恰好提交一次，而在它之后的 NOP 泡泡不会再被错误地送给 Difftest，`this_pc` 差一个指令的错误也随之消失。

6 分支预测与 flush 机制

当前版本的分支预测实现统一采用 Always-Not-Taken 策略：取指阶段总是假定下一条指令位于 `pc + 4`，而不考虑分支或跳转的偏移。真正的分支判断和跳转目标计算发生在 EX 阶段，一旦 EX 得出实际的分支是否跳转以及真正的目标 PC，就会把该结果与预测进行比较。如果预测错误，核心会发出错预测信号，并在下一拍冲刷 IF/ID 和 ID/EX 级的 `valid` 位，同时把取指重定向到正确的 `branch_correct_pc`，从而丢弃错误路径上已经进入流水但尚未提交的指令。虽然策略简单，但整个预测和修正逻辑都被封装在 `predictor_static.sv` 与 EX 阶段之间的接口上，将来替换成基于分支历史表和 BTB 的动态预测器时，只需要在这一小块逻辑附近做修改，而不需要重构整条流水线。

7 mul/div 扩展接口设计

针对后续实验中即将引入的多周期乘除法指令，本次实现并没有直接在 EX 阶段内组合实现 `*` 和 `/` 运算，而是通过 `muldiv_stub.sv` 预留了一个标准化的多周期单元接口。译码阶段可以根据 M 扩展的编码打上 `is_muldiv` 标记，EX 阶段在看到该标记时，不再直接生成结果，而是通过 `req_valid`、`op_a`、`op_b` 和 `op_sel` 向乘除法单元发起请求。乘除法单元则通过 `busy`、`ready` 和 `result_valid` 这些握手信号与主流水线配合，在内部使用寄存器和状态机逐步完成运算。当前占位模块始终返回空闲且无结果，保证 Lab1 阶段不会真正触发乘除法逻辑，但接口已经就位，后续只需替换该模块内部实现即可。

8 验证过程与结果

最终执行 `make test-lab1` 对处理器进行验证。仿真输出中可以看到 NEMU 侧报告 `Core 0: HIT GOOD TRAP at pc = 0x80010004`，并且不再出现以前的 `this_pc different` 报错，也不再出现 GOOD TRAP 之后继续提交 NOP 的现象。